



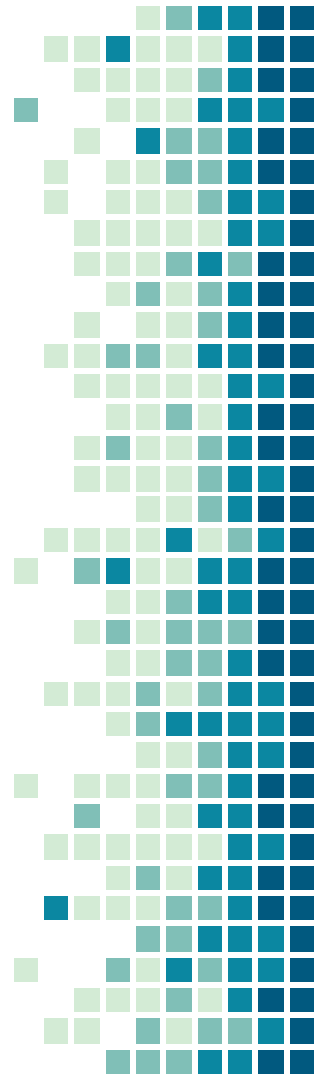
CdL in Informatica – A.A. 2024 – 2025

Programmazione 1 – Modulo 2

Lezione 12
26/11/2024

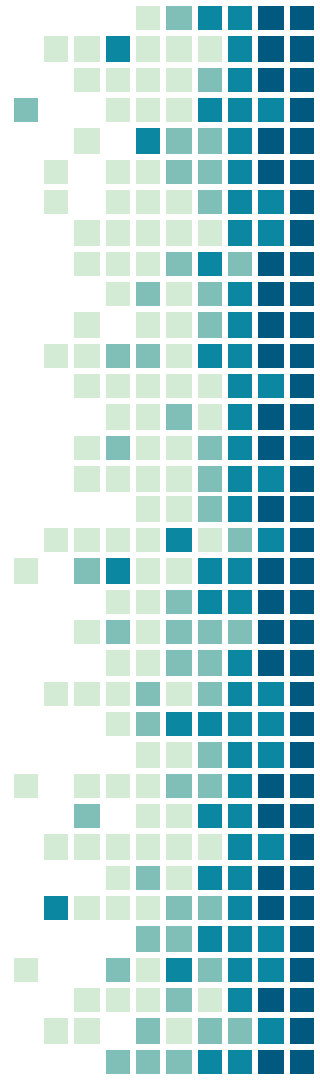
Andrea Loddo

Federico Meloni - Alessandra Perniciano - Fabio Pili

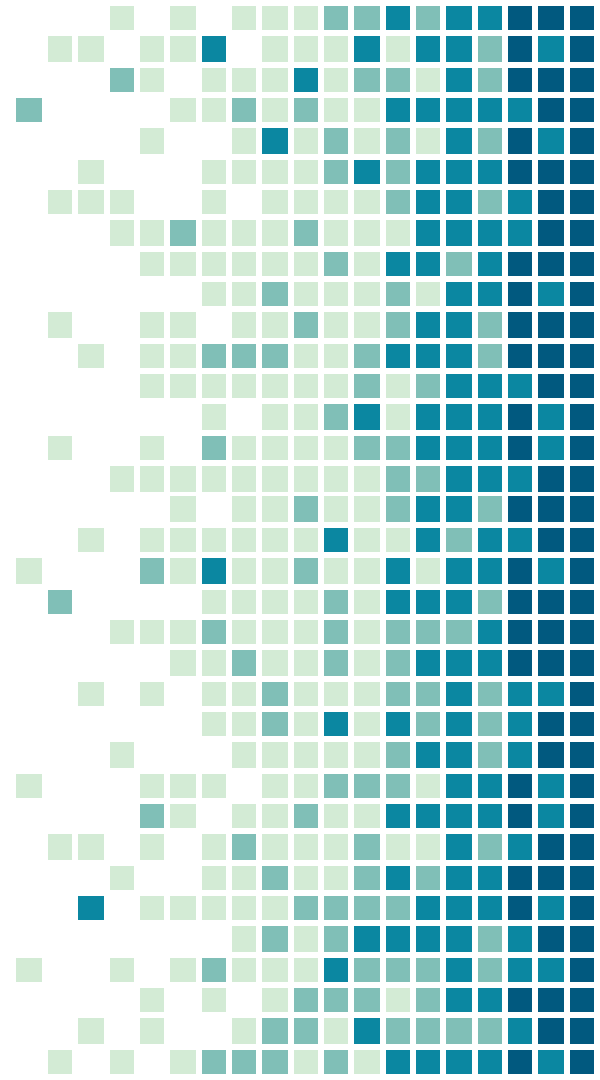


Prossimi episodi

- 28/11: esercitazione su subroutine, strutture, puntatori, allocazione dinamica della memoria
- 03/12: lezione
- 05/12: esercitazione preparatoria al test 2
- 10/12: test 2 (argomenti: tutti fino a questa lezione compresa)



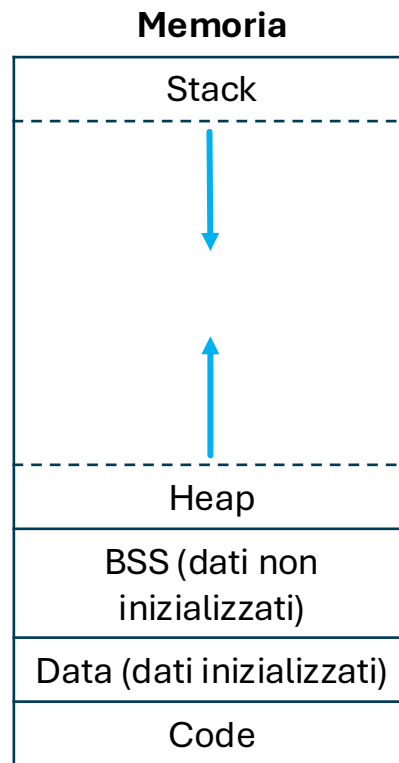
Allocazione dinamica della memoria



Organizzazione della memoria

Un programma eseguibile scritto in linguaggio C prevede la seguente organizzazione della memoria:

- **Stack:** contiene i frame di attivazione delle subroutine, che a loro volta contengono le variabili dichiarate nel loro scope;
- **Heap:** contiene le allocazioni dinamiche;
- **BSS:** contiene variabili o costanti globali non inizializzate;
- **Data:** contiene variabili globali e statiche, che hanno un valore definito e possono essere modificate;
- **Code:** contiene il codice eseguibile.



Allocazione dinamica

- Dichiarare una variabile all'interno del programma = allocare automaticamente o staticamente nella memoria del sistema.
 - Allocazione statica: variabili statiche o globali, effettuata a compile-time, sul Data frame.
 - Allocazione automatica: variabili locali effettuata a compile-time, sullo Stack frame.
- È possibile richiedere al sistema di allocare dinamicamente dello spazio in memoria.
- L'allocazione dinamica è effettuata a runtime, e non a tempo di compilazione, nella porzione di memoria detta Heap.
- Utile:
 - quando non conosciamo a priori l'occupazione di memoria di cui avremo bisogno nel programma;
 - perché permette di fare riferimento a porzioni di memoria che non sono legate allo scope della subroutine in cui sono dichiarate (dichiarare tutto nel main non è l'ideale);
 - permette di lavorare con strutture dati differenti dagli array (es: liste, alberi, ecc): le vedrete in algoritmi e strutture dati al prossimo semestre.
- Richiede l'inclusione di `stdlib.h`.

L'operatore sizeof

L'operatore unario **sizeof** fornisce la dimensione in byte di una certa espressione:

```
int a = 0;  
size_t dim = sizeof(a);
```

Restituisce un valore di tipo **size_t**, che corrisponde a un tipo di dato senza segno (unsigned), dipendente dall'implementazione.

Esempi:

- la funzione `strlen` restituisce un valore **size_t**
- per allocare dinamicamente memoria, occorre indicare dei valori di tipo `size_t`. Vediamo ora come.

Malloc

Prototipo: `void * malloc(size_t size);`

La funzione **malloc** (memory alloc) alloca in memoria un numero di byte pari a **size**, e rende un puntatore al tipo void che indica un generico indirizzo di memoria.

La malloc alloca memoria senza inizializzarla.

Per utilizzare la malloc è necessario conoscere:

- la dimensione che si vuole riservare
- il tipo di dato da allocare, in modo da poter convertire l'indirizzo di memoria restituito in un puntatore a quel tipo di dato.

Esempio:

```
int *a = NULL;
int i, n;

scanf( "%d", &n );
a = (int *)malloc(n * sizeof(int));

if ( a == NULL )
    exit(-1);

for(i=0; i<n; i++)
    a[i] = i;
```

Prima di utilizzare il puntatore a cui assegniamo l'indirizzo restituito dalla malloc, è buona norma verificare che l'allocazione sia andata a buon fine.

a viene utilizzato come un array. Perché?

Calloc

Prototipo: `void * calloc(size_t num, size_t size);`

La funzione `calloc` alloca in memoria **num** elementi consecutivi, ciascuno dei quali ha dimensione `size` (espressa in byte) e pone a zero ciascun bit dello spazio allocato.

La principale differenza con la `malloc` è che i valori sono già inizializzati

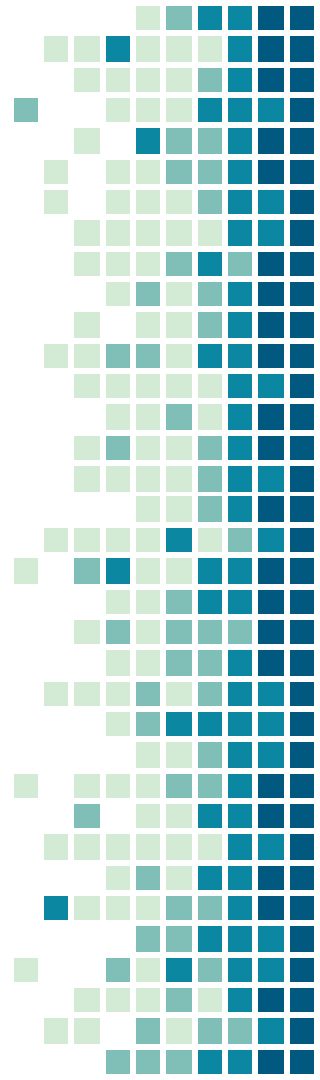
- Notevolmente più efficiente

```
int *a = NULL;
int i, n;

scanf( "%d", &n );
a = (int *)calloc(n, sizeof(int));

if ( a == NULL )
    exit(-1);

for(i=0; i<n; i++)
    printf("%d", a[i]);
```



Realloc

Prototipo:

```
void * realloc(void * ptr, size_t size);
```

Ri-alloca la memoria puntata dal puntatore ptr, portandola alla dimensione size (espressa in byte), preservando il contenuto della memoria (salvo riallocazione con dimensione inferiore).

Utile quando lo spazio che si è allocato in precedenza non è più sufficiente: in questo caso, una realloc evita di dover copiare singolarmente i valori dal vecchio blocco di memoria al nuovo.

Decisamente differente da malloc e calloc: non alloca memoria da zero, ma ha bisogno di un blocco di memoria già allocato.

ATTENZIONE: l'indirizzo di memoria potrebbe cambiare, tenetene traccia!

NB: tutti i controlli sono, come sempre, lasciati al programmatore.

```
int *a = NULL;
int n;
...
scanf("%d", &n);

a = (int *)calloc(n, sizeof(int));
...

n = n + 5;
a = (int *)realloc(a, n * sizeof(int));
```

L'array viene ridimensionato, aggiungendo lo spazio per i nuovi elementi senza cancellare quelli inseriti.

Free

Prototipo:

```
void free(void* ptr);
```

Il C non ha un garbage collector, cioè un sistema automatico che “recupera” a runtime la memoria non più referenziata.

La memoria non più referenziata da alcun puntatore non è più raggiungibile e quindi utilizzabile: **memory leak**.

È compito del programmatore liberare (de-allocare) esplicitamente la memoria allocata in precedenza, quando non è più utile.

La **free** dealloca lo spazio puntato dal puntatore ptr, precedentemente allocato.

- Funziona soltanto se lo spazio puntato da ptr è stato allocato dinamicamente, altrimenti genera un errore a runtime.

```
int *a = NULL;
int i, n;

scanf( "%d", &n );
a = (int *)malloc(n * sizeof(int));
...
if ( a != NULL )
    free(a);
```

Allocazione dinamica di matrici

Esempio: vogliamo stampare una matrice di dimensione $R \times C$ mediante una procedura generica. Possiamo farlo con gli strumenti attuali?

No: dobbiamo SEMPRE precisare almeno la seconda dimensione di una matrice allocata staticamente.

```
void stampaMatrice( int r, int c, int mat[][c] );
```

Possiamo allocare dinamicamente la matrice?

- Come è memorizzata una matrice in memoria?

Due modalità possibili:

1. Allochiamo uno spazio contiguo di $r \times c$ celle, e "simuliamo" l'accesso alla matrice in maniera sequenziale.

```
/* Allocazione dinamica di una matrice, esempio 1 */
int r = 4, c = 3;
int *matrice = NULL;
int i, j;
matrice = (int*) calloc( r*c, sizeof(int) );

for(i = 0; i < r; i++) {
    for (j = 0; j < c; j++) {
        printf("%d ", matrice[i*c + j]);
    }
    printf("\n");
}
```

Allocazione dinamica di matrici

Due modalità possibili:

2. Allochiamo uno spazio contiguo di r righe e, per ogni riga, allochiamo c colonne.

```
/* Allocazione dinamica di una matrice, esempio 2 */
int r = 4, c = 3;
int **matrice = NULL;
int i, j;
matrice = (int**) calloc(r, sizeof(int*));
for(i=0; i<r; i++) {
    matrice[i] = (int*) calloc(c, sizeof(int));
}
```

```
/* Stampa di una matrice allocata
dinamicamente */

for(i=0; i < r; i++) {
    for (j = 0; j < c; j++) {
        printf("%d ", matrice[i][j]);
    }
    printf("\n");
}
```

Sfruttiamo il fatto che, in C, una matrice è un **array di righe**.

Di conseguenza: l'array i -esimo è un puntatore all'array contenente le celle di quella riga.

Per questo motivo, abbiamo bisogno di un **puntatore a puntatore** per tenerne traccia.

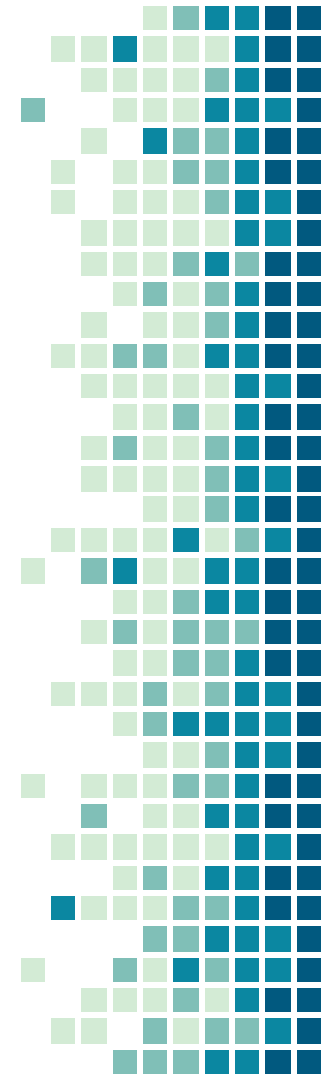
Come possiamo liberare la memoria allocata dinamicamente in questo caso?

FINE!

Domande?

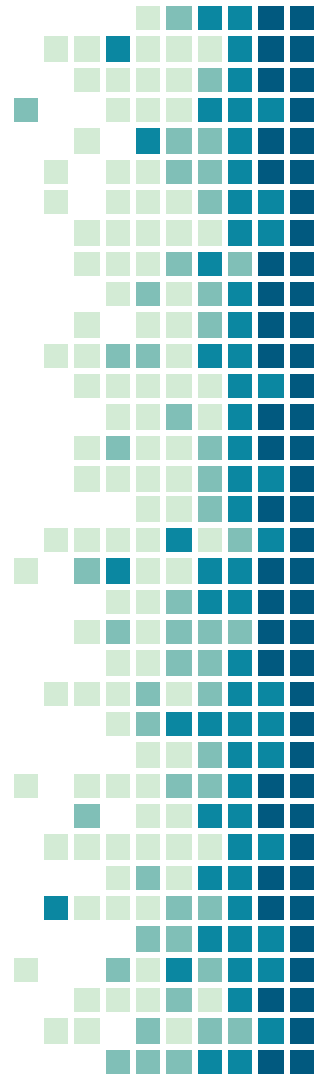
Autovalutazione

1. A cosa serve la malloc?
2. A cosa serve la calloc?
3. Che differenza c'è tra malloc e calloc?
4. Perché è necessario inizializzare un puntatore a NULL?
5. Che differenza c'è tra allocare un array staticamente e allocare un array dinamicamente?
6. Che tipo di puntatore restituiscono la calloc e la malloc?
7. A cosa serve la funzione free?
8. Che parametri necessitano le funzioni malloc, calloc e free?
9. Com'è rappresentata una matrice in memoria?
10. Posso allocare dinamicamente una matrice? Con quali modalità?
11. Cos'è un puntatore a puntatore?



Esercizi

1. Scrivere una funzione "createArray(int n, int initialValue)" che allochi dinamicamente un array di n elementi inizializzati a initialValue, e restituisca il puntatore al primo elemento dell'array.
2. Scrivere una funzione "duplicate" che utilizzi l'allocazione dinamica per creare una copia di una stringa.
3. Scrivere una funzione che calcoli la media degli elementi di un array allocato dinamicamente.
4. Definire una struttura 'Albergo' composta dai campi 'numeroStanze', 'prezzo', 'localita' che permetta di acquisire i relativi dati attraverso una procedura passando come parametro l'indirizzo di memoria della stessa. Scrivere poi una seconda procedura che ne permetta la stampa. La struttura va allocata dinamicamente nel main.
5. Scrivere una funzione che verifichi se c'è una sovrapposizione tra due intervalli (min1,max1) e (min2,max2). Se sì, la funzione restituirà vero e il punto medio della loro sovrapposizione si troverà in un argomento di output, altrimenti renderà falso. I due intervalli sono quattro valori allocati dinamicamente e vanno acquisiti mediante input utente.
6. Scrivere un programma in cui viene allocato dinamicamente un vettore di dimensione scelta in input dall'utente. Il vettore deve essere popolato con numeri interi, positivi e negativi, generati casualmente. Scrivere poi una subroutine che riceve come parametro il vettore e inverte l'ordine degli elementi. Visualizzare dal main sia il vettore originale che quello ottenuto dopo la chiamata della funzione.



Esercizi

8. Definire una struttura per la modellazione dei dati anagrafici di uno studente (nome, cognome, data di nascita ecc), quindi creare e popolare un array di studenti. Aggiungere una seconda struttura per la modellazione dei risultati di una sessione d'esame, che contiene, oltre alla data, al luogo e al nominativo del professore, un elenco di voti abbinati agli studenti (suggerimento: utilizzare gli indici degli studenti nel primo array per recuperare le informazioni). Scrivere una funzione che verifica che lo stesso studente non compaia due volte nell'elenco, un'altra funzione che conta quanti studenti hanno sostenuto l'esame con esito positivo, ed un'altra funzione che visualizza i nomi e i voti degli studenti che hanno ottenuto voto maggiore o uguale a 28. Inserire gli Studenti in un array dichiarato dinamicamente. La dimensione dell'array deve crescere ad ogni inserimento di un nuovo studente.
9. Scrivere una funzione creaMatriceDinamica che riceva due interi R, C, in ingresso rappresentanti il numero di righe e il numero di colonne, e restituisca una matrice RxC allocata dinamicamente.
10. Scrivere un programma che crei dinamicamente una matrice usando la funzione dell'esercizio 9, la popoli casualmente e la stampi in output.
11. Scrivere un programma che permetta di visualizzare la k-esima stringa tra le n inserite dall'utente in un array di stringhe. Chiedere inizialmente all'utente quante stringhe vuole inserire, dopodiché acquisirle una alla volta. L'array deve essere allocato dinamicamente.
12. Scrivere una funzione che concateni tutte le stringhe presenti nell'array dell'esercizio precedente. La stringa risultante deve essere allocata dinamicamente, e la sua lunghezza deve essere esattamente pari al numero di caratteri che contiene (più il carattere `\0`). La funzione deve restituire poi il puntatore al primo carattere della stringa.

